Líder en Ciencia y Tecnología

UNIVERSIDAD NACIONAL DE INGENIERÍA

Área de Conocimiento DACTIC
Ingeniería de Sistemas

PROYECTO FINAL DE CURSO

Sistema Web para Control de Operaciones del Taller “El Guatuso”

INTEGRANTES DEL EQUIPO

- Br. Alejandro Enrique Guido Masis
- Br. Josué David Beteta Guadamuz
- Br. Lenin Camilo Sequeira Palacios

Diseño de Sistemas de Internet

Docente: **Ing. Cristopher Larios**

Grupo: 5T1-IS • Noviembre 2026

Taller de Amortiguadores “El Guatuso”

! Situación Anterior (Manual)

- **Registros en papel:** Cuadernos y hojas de Excel aisladas.
- **Sin historial ágil:** Lentitud al buscar reparaciones por placa.
- **Incertidumbre de stock:** Descontrol de repuestos en bodega.
- **Cierres manuales:** Errores de cálculo en presupuestos y caja.

4 Perfiles Conectados en Taller+

- **Cajero:** Recepción de clientes, presupuestos, cobros y caja diaria.
- **Mecánico:** Avance de órdenes desde el móvil e informe técnico.
- **Supervisor:** Coordinación de taller, stock y arqueos.
- **Administrador:** Catálogos, personal, proveedores y finanzas.

Objetivo General y Objetivos Específicos

OBJETIVO GENERAL

Desarrollar un sistema web de control operativo para el taller “El Guatuso” integrando los módulos de servicios, órdenes, presupuestos, inventario y clientes para optimizar la atención y el registro de datos.

01

Modelado UWE

Analizar requerimientos y diseñar diagramas UWE del sistema.

02

Flujo Mecánico

Seguimiento completo desde el presupuesto hasta la entrega.

03

Base de Datos

SQL Server estructurado con procedimientos almacenados.

04

Reportes y Nube

Control de ingresos, caja diaria y despliegue en Railway.

Cumplimiento de Requerimientos y Adaptaciones Prácticas

12 **Requerimientos Funcionales (RF)**
Clientes, Vehículos, Servicios, Packs, Presupuestos, Órdenes, Finanzas y Personal implementados.

06 **Requerimientos No Funcionales (RNF)**
Respuestas < 5s, cifrado SHA-256, acceso por 4 roles, diseño Responsive y arquitectura modular.

RF-10 • Filtro Financiero

Se adaptó a un selector directo por **Año, Mes y Día** en vez de dos calendarios, porque en el taller las cuentas se cierran por día de caja o por mes.

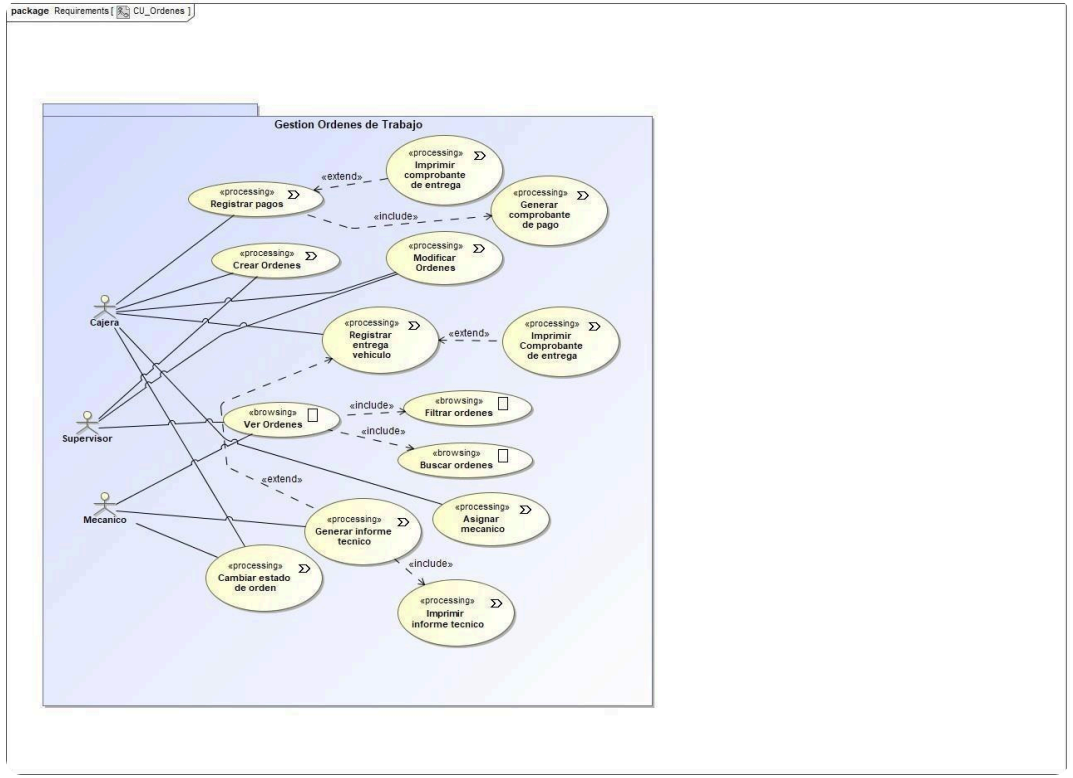
RF-09 • Informe Técnico

Queda guardado en el expediente de **Entrega de la Orden** y se imprime directo desde el navegador, sin llenar de archivos la PC de recepción.

RNF-02 • Claves de Taller

Claves **100% encriptadas (SHA-256)**, permitiendo claves ágiles (6+ caracteres) a los mecánicos para facilitar su acceso rápido desde el celular.

Modelado UWE: Casos de Uso y Diagrama de Contenido



Casos de Uso UWE — Gestión de Órdenes de Trabajo

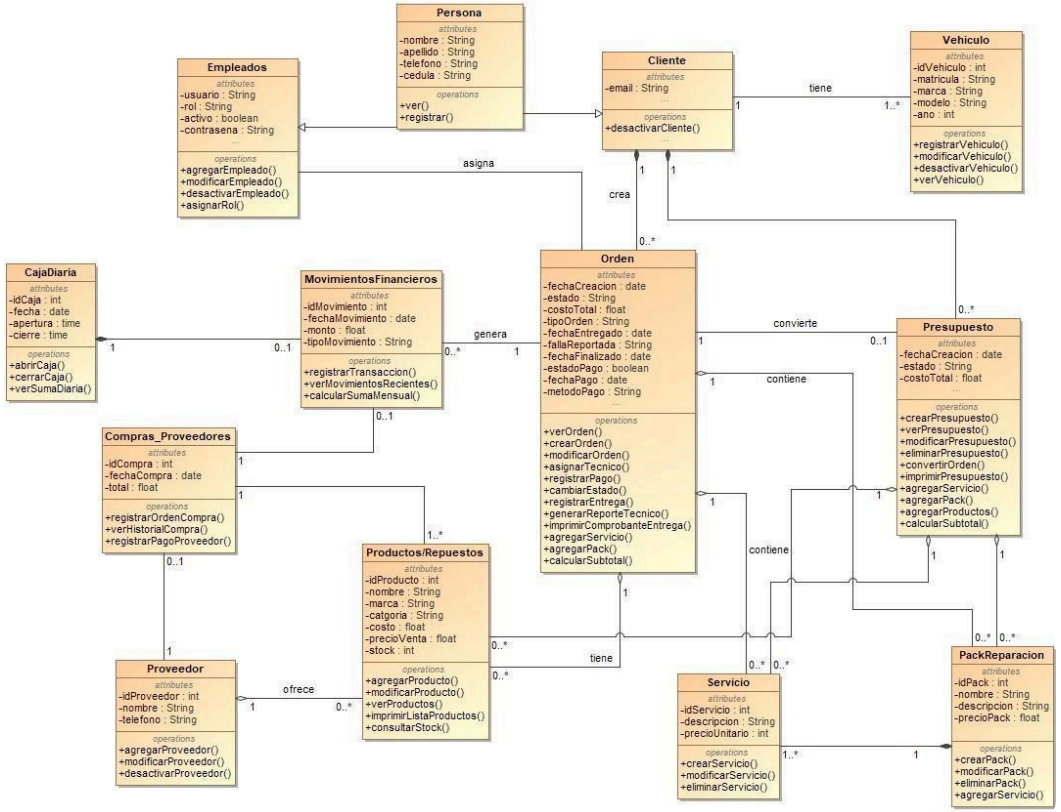


Diagrama de Contenido UWE — Entidades del Dominio

Modelado UWE: Navegación por Rol y Presentación

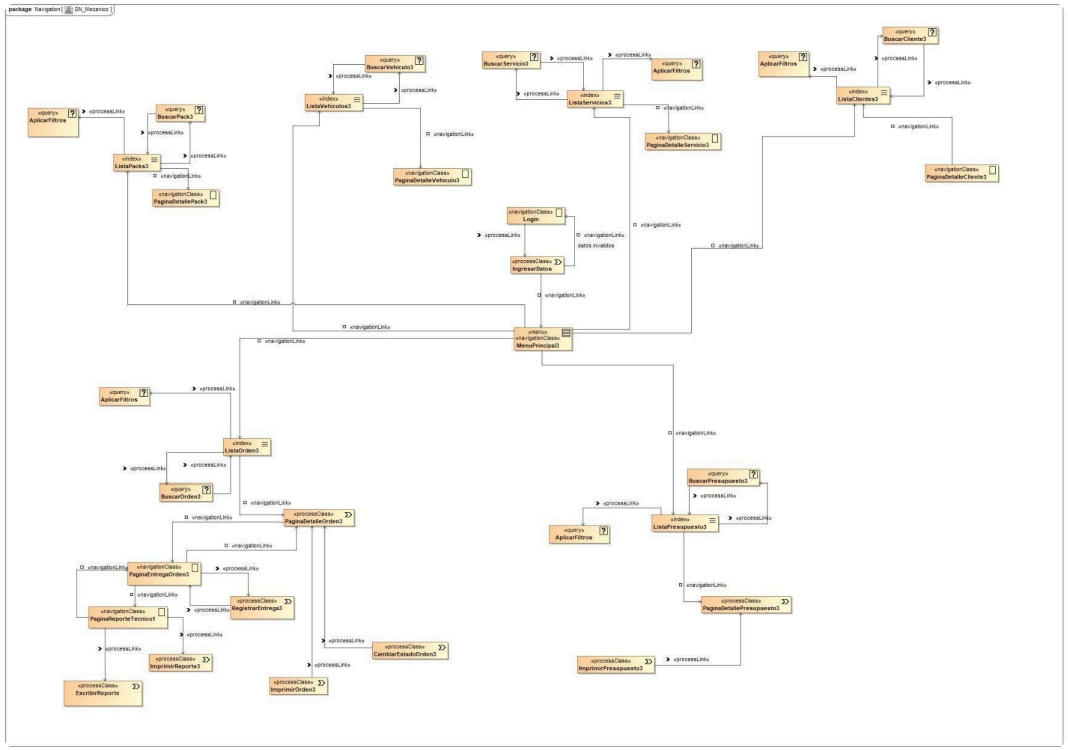


Diagrama Navegacional UWE — Flujo por Perfil Operativo

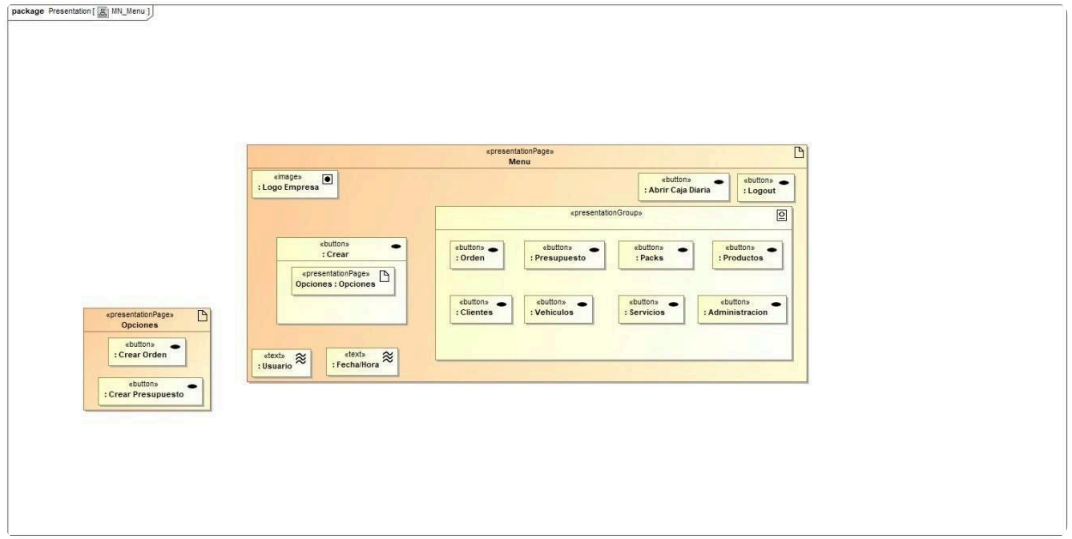
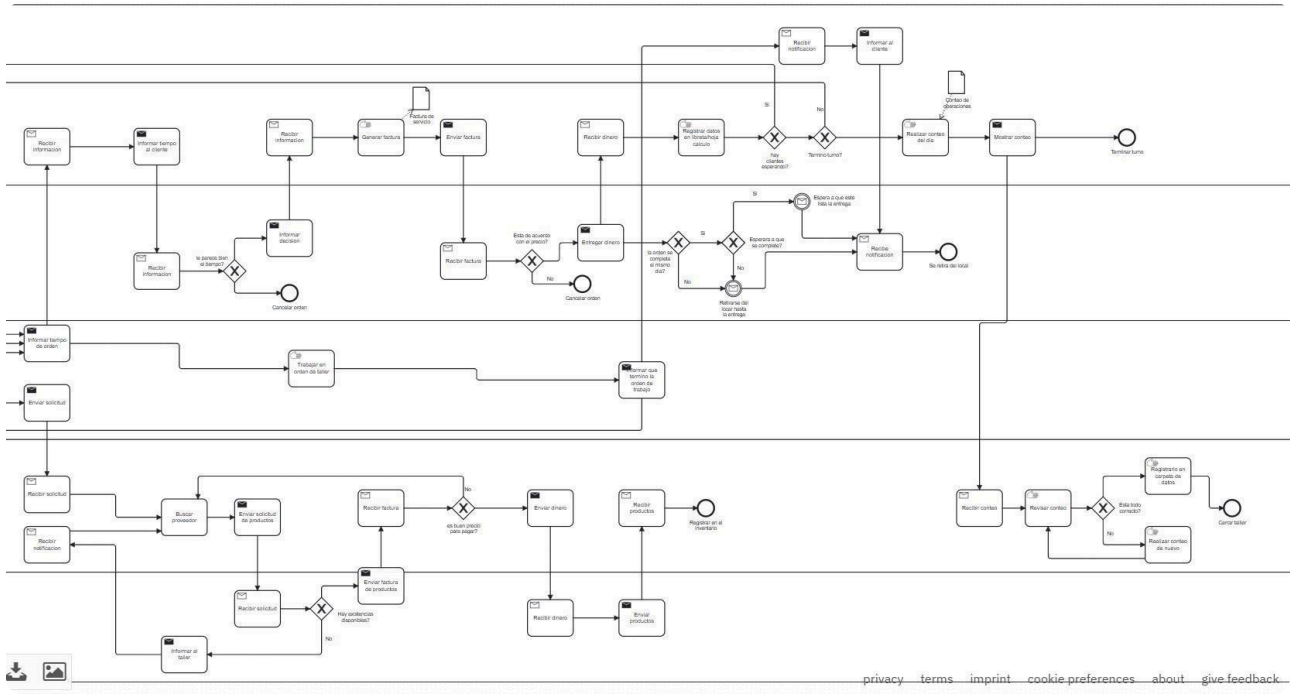


Diagrama de Presentación UWE — Estructura del Menú Principal

CAPÍTULO I

Modelado de Procesos del Taller (Notación BPMN)

UNI



Flujo BPMN de Recepción, Diagnóstico, Verificación de Stock y Ejecución de Orden

1. Recepción

Registro de cliente y vehículo

2. Presupuesto

Cotización de servicios y packs

3. Reparación

Descuento de stock y trabajo

4. Entrega y Caja

Informe técnico y pago

Arquitectura Elegida: Cliente-Servidor en Capas + MVC

UNI

M Modelo (Datos)

- **31 entidades** en C# y `BdTallerContext`.
- Reglas de inventario, subtotales y caja en **SQL Server**.

V Vista (Interfaz)

- Pantallas **Razor (`.cshtml`)** y Bootstrap 5.
- Clientes ligeros para PC de recepción y móviles de mecánicos.

C Controlador (Lógica)

- **15 controladores** organizados por módulo.
- Validan permisos por rol y orquestan las peticiones HTTP.

¿Por qué se eligió frente a SPA o Microservicios?

Garantiza **consistencia inmediata (ACID)** entre órdenes, stock y caja con menor costo operativo, dejando lista la ruta de migración futura a microservicios mediante contenedores Docker.

Tecnologías e Ingeniería del Backend

1 C# y ASP.NET Core 8.0 MVC

Lenguaje fuertemente tipado y orientado a objetos con **Inyección de Dependencias nativa**, logrando bajo acoplamiento entre los 15 módulos del taller.

2 EF Core 9 + Stored Procedures

Enfoque híbrido: consultas asíncronas rápidas con LINQ y transacciones críticas (stock, órdenes y arqueos) mediante **Procedimientos Almacenados en SQL Server**.

3 Seguridad: Cookies + SHA-256

Autenticación encriptada basada en **Claims**, contraseñas cifradas en binario (`VARBINARY`), bloqueo por intentos fallidos y tokens de recuperación.

4 Control de Roles Centralizado

Clase `RolesSistema.cs` y atributos `[Authorize]` que protegen cada acción según el perfil: **Administrador, Supervisor, Cajero y Mecánico**.

Tecnologías Frontend e Integración con el Backend

UI Vistas Razor + Bootstrap 5

- **14 carpetas de vistas (`.cshtml`)** renderizadas en el servidor.
- Diseño **100% Responsive** con tipografía *Outfit* e iconografía vectorial clara.

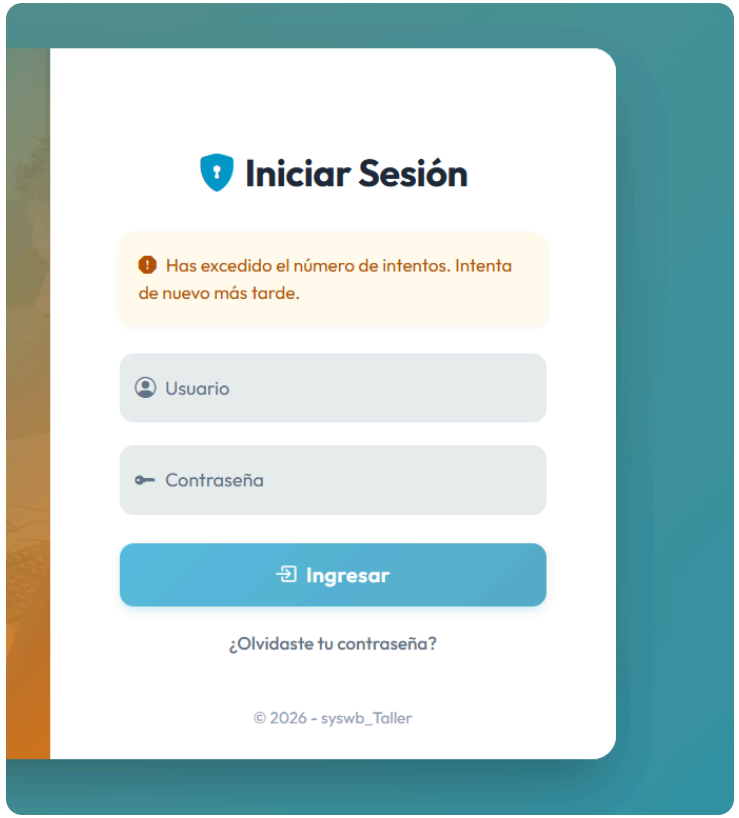
✓ Validación en Tiempo Real

- **jQuery + `intl-tel-input`** para validar teléfonos y campos obligatorios antes de enviar al servidor.
- Alertas inmediatas ante falta de stock en bodega.

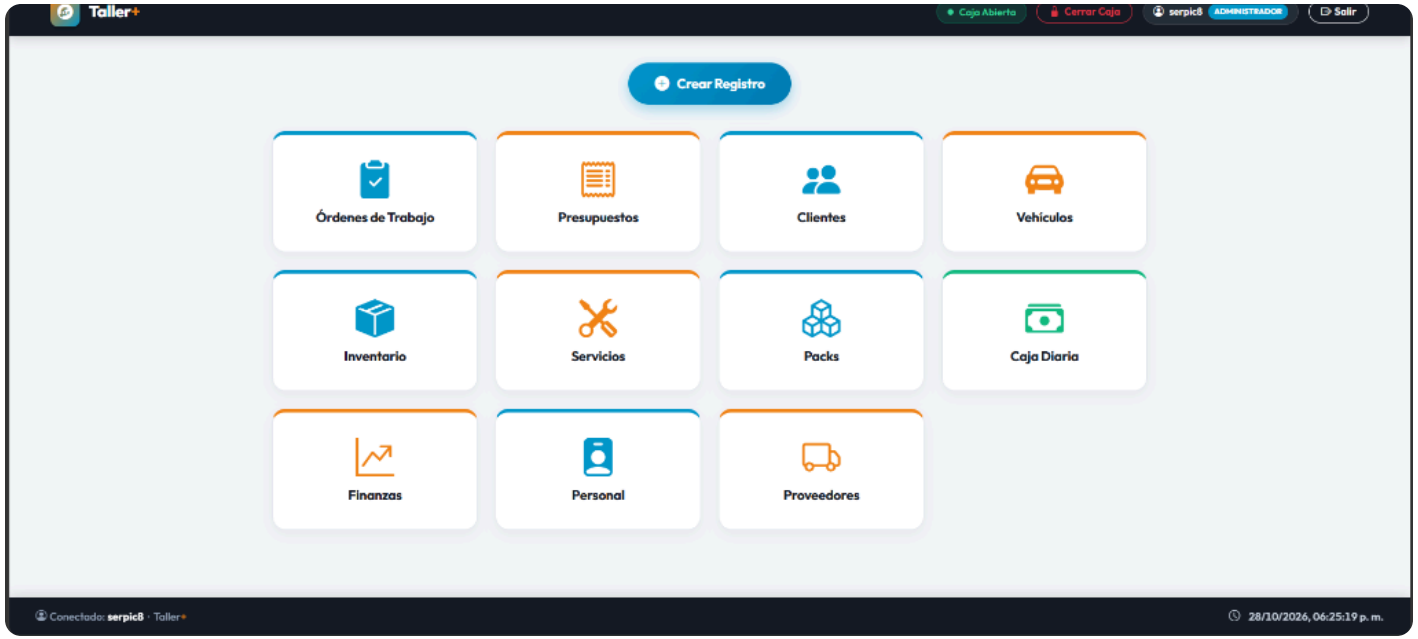
⇄ Integración Destacada: Header Global con Estado de Caja en Vivo

La plantilla maestra (`_Layout.cshtml`) se integra en tiempo real con la base de datos para mostrar permanentemente la píldora **"Caja Abierta"** o **"Caja Cerrada"** y adaptar los botones visibles según el rol del empleado.

Evidencias Frontend: Control de Acceso y Menú Principal



Inicio de Sesión con Protección de Intentos



Menú Principal de Taller+ con Indicador “Caja Abierta” y Rol

Despliegue en la Nube y Seguimiento de Cambios (CI/CD)

01

Git & GitHub

Control de versiones en la rama `master` tras auditar el código.

02

GitHub Actions

Pipeline automático (`ci-cd.yml`) autenticado con `RAILWAY_TOKEN`.

03

Docker .NET 8

`Dockerfile` multi-etapa optimizado en modo Release (Puerto 8080).

04

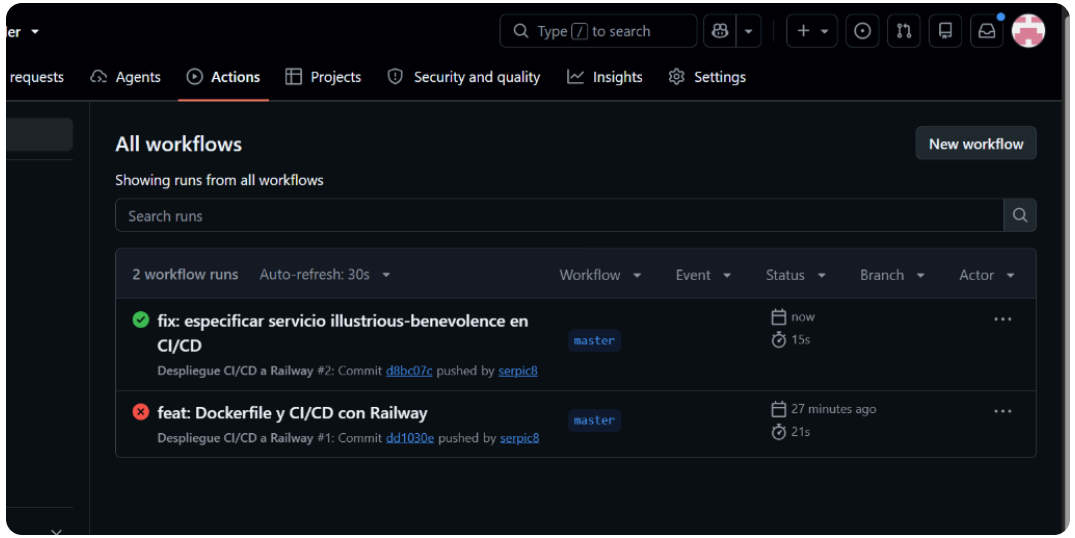
Railway + SQL

App Web conectada por red privada al contenedor **SQL Server 2022**.

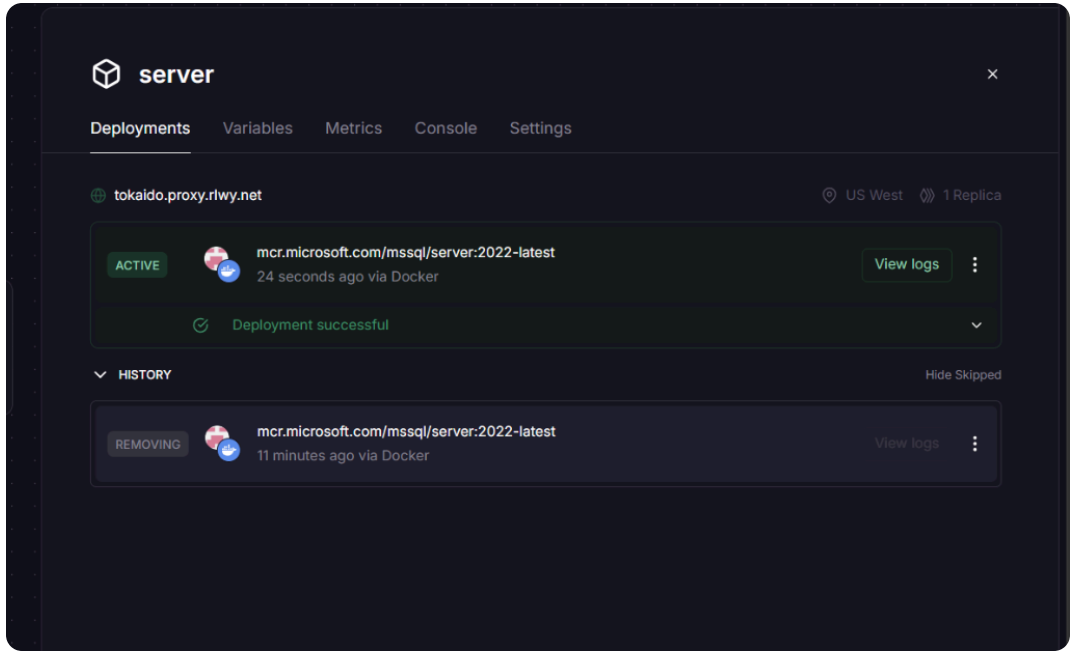
Inicialización Automática de Base de Datos (DatabaseInitializer.cs)

Al iniciar en Railway, el sistema normaliza automáticamente el cifrado TLS y la IP interna IPv4, verifica si existe `BD_Taller` y ejecuta `BD_Taller_init.sql` sin intervención manual.

Evidencias de Despliegue: GitHub Actions y SQL Server en Railway

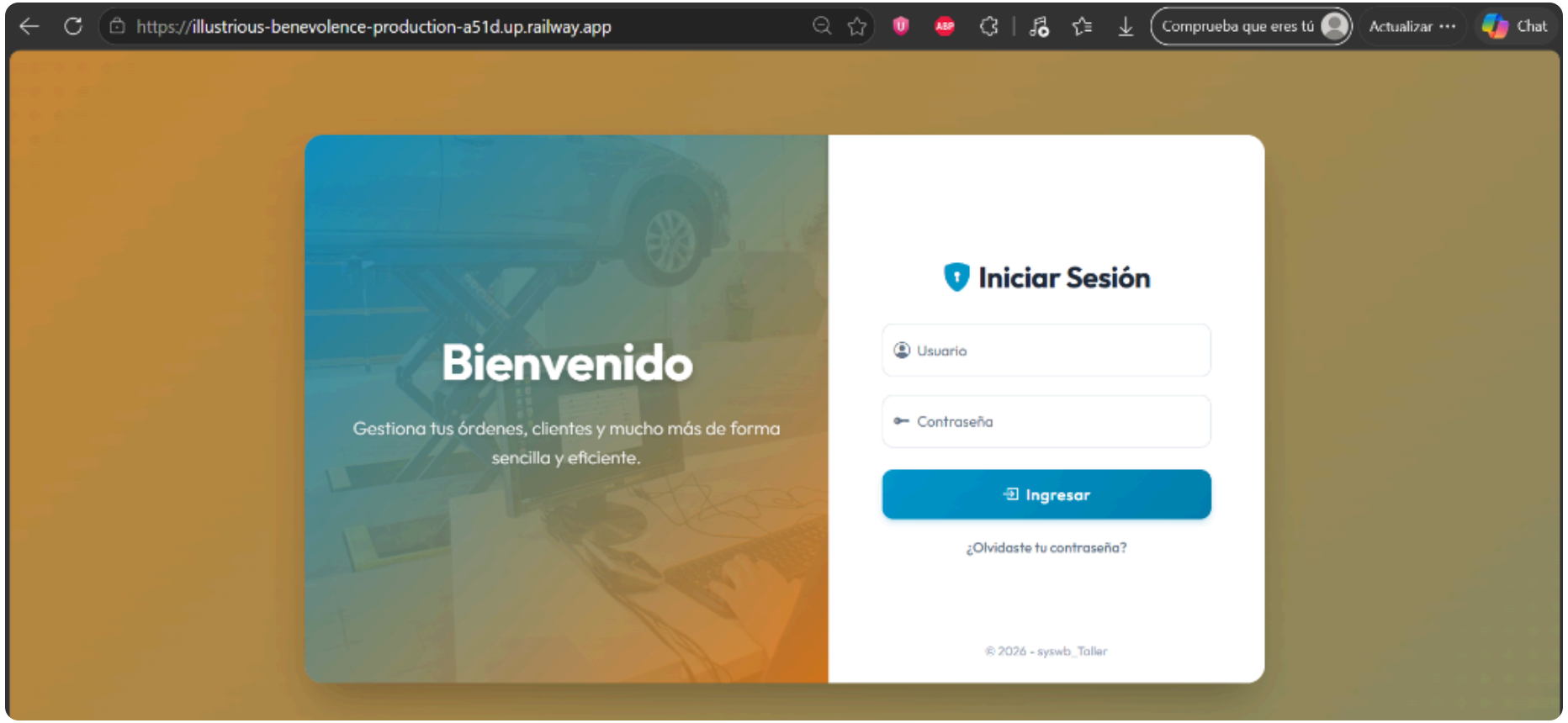


Pipeline CI/CD Ejecutado con Éxito en GitHub Actions



Contenedor SQL Server 2022 Activo en Railway Cloud

Sistema Taller+ Operando en Producción (Dominio Público HTTPS)



https://illustrious-benevolence-production-a51d.up.railway.app

Conclusiones y Recomendaciones

✓ Conclusiones

- **Digitalización total:** Se reemplazó el uso de cuadernos por un flujo conectado entre recepción, bodega y caja.
- **Fidelidad operativa:** Los modelos UWE y las adaptaciones prácticas facilitaron el uso diario del personal.
- **Integridad contable:** SQL Server y EF Core aseguran exactitud en stock e ingresos.
- **Despliegue moderno:** Automatización completa en la nube con Docker, GitHub Actions y Railway.

→ Recomendaciones

- **Respaldos semanales:** Programar backups periódicos de la base de datos `BD_Taller`.
- **Capacitación por rol:** Entrenamiento práctico breve para cajeros y mecánicos.
- **Avisos por WhatsApp:** Integrar en el futuro alertas automáticas al finalizar una orden.
- **Continuidad CI/CD:** Mantener el flujo de pruebas antes de cada actualización a `master`.



UNIVERSIDAD NACIONAL DE INGENIERÍA

Diseño de Sistemas de Internet • 5T1-IS

DEFENSA DE PROYECTO FINAL

¡Muchas Gracias por su Atención!

REPOSITORIO EN GITHUB

`https://github.com/serpic8/syswb_Taller.git`

SISTEMA DESPLEGADO EN RAILWAY

`https://illustrious-benevolence-production-a51d.up.railway.app`

EXPOSITORES

- **Br. Alejandro Enrique Guido Masis**
- **Br. Josué David Beteta Guadamuz**
- **Br. Lenin Camilo Sequeira Palacios**